

A decorative graphic consisting of a thin yellow circle on the left side. A horizontal bar, divided into a yellow left half and a light gray right half, passes through the center of the circle. Large black and yellow brackets are positioned on the left and right ends of the bar, respectively.

Building and Running Modules

Linux Kernel Programming

[Role of a Module]

- Dynamically add kernel functionality
 - Modularized code running in kernel space
 - Does not require reboot
- Out of tree drivers can be easily included
- Kernel image size can be kept small

[Setting Up Your Test System]

- Requirements for building/using external modules
 - configuration
 - kernel header files
 - kernel built with “modules enabled”
 - `[*] Enable loadable module support --->`
- **make modules_prepare**
 - will not build Module.symvers for module versioning

[The Hello World Module]

```
#include <linux/init.h>
#include <linux/module.h>
MODULE_LICENSE("Dual BSD/GPL");

static int hello_init(void) {
    printk(KERN_ALERT "Hello, world\n");
    return 0;
}

static void hello_exit(void) {
    printk(KERN_ALERT "Goodbye, cruel world\n");
}

module_init(hello_init);
module_exit(hello_exit);
```

No main function

The Hello World Module

```
#include <linux/init.h>
#include <linux/module.h>
MODULE_LICENSE("Dual BSD/GPL");
```

```
static int hello_init(void) {
    printk(KERN_ALERT "Hello, world\n");
    return 0;
}
```

Invoked when the module is loaded

```
static void hello_exit(void) {
    printk(KERN_ALERT "Goodbye, cruel world\n");
}
```

```
module_init(hello_init);
module_exit(hello_exit);
```

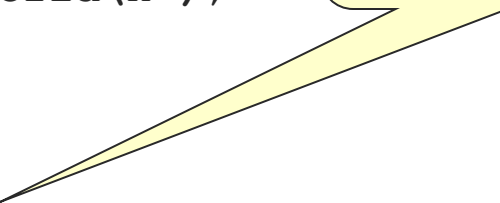
[The Hello World Module]

```
#include <linux/init.h>
#include <linux/module.h>
MODULE_LICENSE("Dual BSD/GPL");

static int hello_init(void) {
    printk(KERN_ALERT "Hello, world\n");
    return 0;
}

static void hello_exit(void) {
    printk(KERN_ALERT "Goodbye, cruel world\n");
}

module_init(hello_init);
module_exit(hello_exit);
```



Invoked when the module is removed

The Hello World Module

```
#include <linux/init.h>
#include <linux/module.h>
MODULE_LICENSE("Dual BSD/GPL");

static int hello_init(void) {
    printk(KERN_ALERT "Hello, world\n");
    return 0;
}

static void hello_exit(void) {
    printk(KERN_ALERT "Goodbye, cruel world\n");
}

module_init(hello_init);
module_exit(hello_exit);
```

Macros to indicate which module initialization and exit functions to call

[The Hello World Module]

```
#include <linux/init.h>
#include <linux/module.h>
MODULE_LICENSE("Dual BSD/GPL");

static int hello_init(void) {
    printk(KERN_ALERT "Hello, world\n");
    return 0;
}

static void hello_exit(void) {
    printk(KERN_ALERT "Goodbye, cruel world\n");
}

module_init(hello_init);
module_exit(hello_exit);
```

This module bears
a free license

The Hello World Module

```
#include <linux/init.h>
#include <linux/module.h>
MODULE_LICENSE("Dual BSD/GPL");

static int hello_init(void) {
    printk(KERN_ALERT "Hello, world\n");
    return 0;
}

static void hello_exit(void) {
    printk(KERN_ALERT "Goodbye, cruel world\n");
}

module_init(hello_init);
module_exit(hello_exit);
```

The ordering
matters sometimes



[The Hello World Module]

```
#include <linux/init.h>
#include <linux/module.h>
MODULE_LICENSE("Dual BSD/GPL");
```

```
static int hello_init(void) {
    printk(KERN_ALERT "Hello, world\n");
    return 0;
}
```

~= printf in C library

```
static void hello_exit(void) {
    printk(KERN_ALERT "Goodbye, cruel world\n");
}
```

```
module_init(hello_init);
module_exit(hello_exit);
```

The Hello World Module

```
#include <linux/init.h>
#include <linux/module.h>
MODULE_LICENSE("Dual BSD/GPL");
```

```
static int hello_init(void) {
    printk(KERN_ALERT "Hello, world\n");
    return 0;
}
```

```
static void hello_exit(void) {
    printk(KERN_ALERT "Goodbye, cruel world\n");
}
```

```
module_init(hello_init);
module_exit(hello_exit);
```

Indicates the message priority

Note that no ',' after
KERN_ALERT

[Preliminaries]

- Just about all module code includes the following header files
 - `<linux/module.h>`
 - Symbols and functions needed by modules
 - `<linux/init.h>`
 - Allows you to specify initialization and cleanup functions

Initialization and Shutdown

- Initialization function
 - Registers any *facility*, or functionality offered by the module

```
static int __init initialization_function(void) {  
    /* initialization code here */  
}  
module_init(initialization_function);
```

Initialization and Shutdown

- Initialization function
 - Registers any facility, or functionality offered by the module

```
static int __init initialization_function(void) {  
    /* initialization code here */  
}  
module_init(initialization_function);
```

Indicates that the module loader can drop this function after the module is loaded, making its memory available

Initialization and Shutdown

- Initialization function
 - Registers any facility, or functionality offered by the module

```
static int __init initialization_function(void) {  
    /* initialization code here */  
}  
module_init(initialization_function);
```

Mandatory to specify the initialization function

[The Cleanup Function]

- Unregisters various functionalities and returns all resources

```
static void __exit cleanup_function(void) {  
    /* Cleanup code here */  
}  
module_exit(cleanup_function);
```


[The Cleanup Function]

- Unregisters various functionalities and returns all resources

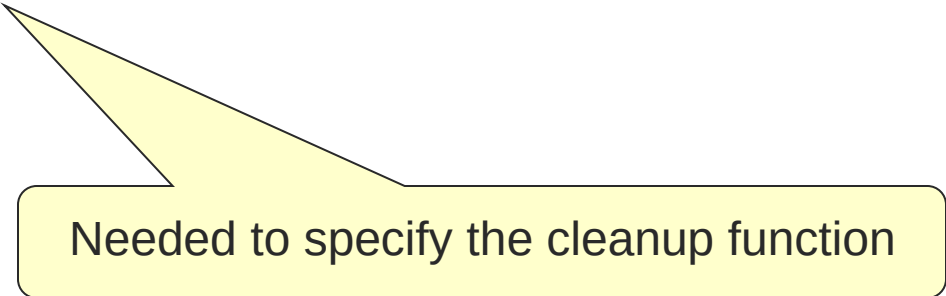
```
static void __exit cleanup_function(void) {  
    /* Cleanup code here */  
}  
module_exit(cleanup_function);
```

Indicates that this function is for unloading only

[The Cleanup Function]

- Unregisters various functionalities and returns all resources

```
static void __exit cleanup_function(void) {  
    /* Cleanup code here */  
}  
module_exit(cleanup_function);
```



Needed to specify the cleanup function

Error Handling During Initialization

```
static int __init my_init_function(void) {
    int err;

    /* registration takes a pointer and a name */
    err = register_this(ptr1, "skull");
    if (err) goto fail_this;
    err = register_that(ptr2, "skull");
    if (err) goto fail_that;
    err = register_those(ptr3, "skull");
    if (err) goto fail_those;

    return 0; /* success */

fail_those:  unregister_that(ptr2, "skull");
fail_that:  unregister_this(ptr1, "skull");
fail_this:  return err; /* propagate the error */
}
```

Error Handling During Initialization

```
static int __init my_init_function(void) {
    int err;

    /* registration takes a pointer and a name */
    err = register_this(ptr1, "skull");
    if (err) goto fail_this;
    err = register_that(ptr2, "skull");
    if (err) goto fail_that;
    err = register_those(ptr3, "skull");
    if (err) goto fail_those;

    return 0; /* success */

fail_those:  unregister_that(ptr2, "skull");
fail_that:  unregister_this(ptr1, "skull");
fail_this:  return err; /* propagate the error */
}
```

Check `<linux/errno.h>` for error codes. Error codes should be a **negative** integer, otherwise the kernel will load the module.

x86 Error Codes

- **<linux/errno.h>**

- (include/linux/errno.h)

- **<uapi/linux/errno.h>**

- (include/uapi/linux/errno.h)

- **<asm/errno.h>**

- (arch/x86/include/uapi/asm/errno.h)

- **uapi/asm-generic/errno.h**

- (include/uapi/asm-generic/errno.h)

- **<asm/errno.h>**

- (arch/x86/include/uapi/asm/errno.h)

- **<asm-generic/errno.h>**

- (include/uapi/asm-generic/errno.h)

- **<asm-generic/errno-base.h>**

- (include/uapi/asm-generic/errno-base.h)

[Goto?]

- Cleaner code for error recovery
- Faster than separate error-handling functions
- Better for the cache
- Great online discussion
 - <http://kerneltrap.org/node/553/2131>

[Cleanup Function]

```
static void __exit my_cleanup_function(void) {  
    unregister_those(ptr3, "skull");  
    unregister_that(ptr2, "skull");  
    unregister_this(ptr1, "skull");  
    return err;  
}
```

[Other Code Patterns]

```
int __init my_init(void) {
    int err = -ENOMEM;
    item1 = allocate_thing(arg1);
    item2 = allocate_thing2(arg2)
    if (!item1 || !item2) goto fail;
    err = register_stuff(item1, item2);
    if (!err) {
        stuff_ok = 1;
    } else {
        goto fail;
    }

    return 0;

fail:
    my_cleanup();
    return err;
}
```


[Other Code Patterns]

```
void my_cleanup(void) {  
    if (item1) release_thing(item1);  
    if (item2) release_thing2(item2);  
    if (stuff_ok) unregister_stuff();  
    return;  
}
```

- No `__exit` when it is called by nonexit code

[Module Loading/Unloading]

```
% make -C /usr/src/linux-3.10.40 M=`pwd` modules
```

Notice the backticks ``
(not single quotes)

[Module Loading/Unloading]

```
% make -C /usr/src/linux-3.10.40 M=`pwd` modules
make: Entering directory `/usr/src/linux-3.10.40'
  Building modules, stage 2.
  MODPOST 1 modules
make: Leaving directory `/usr/src/linux-3.10.40'
% su
Password:
```

[Module Loading/Unloading]

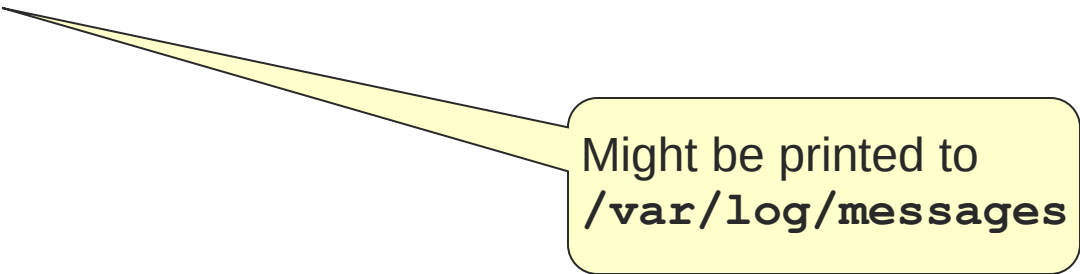
```
% make -C /usr/src/linux-3.10.40 M=`pwd` modules
make: Entering directory `/usr/src/linux-3.10.40'
  Building modules, stage 2.
  MODPOST 1 modules
make: Leaving directory `/usr/src/linux-3.10.40'
% su
Password:
root#
```

[Module Loading/Unloading]

```
% make -C /usr/src/linux-3.10.40 M=`pwd` modules
make: Entering directory `/usr/src/linux-3.10.40'
  Building modules, stage 2.
  MODPOST 1 modules
make: Leaving directory `/usr/src/linux-3.10.40'
% su
Password:
root# insmod hello.ko
```

[Module Loading/Unloading]

```
% make -C /usr/src/linux-3.10.40 M=`pwd` modules
make: Entering directory `/usr/src/linux-3.10.40'
  Building modules, stage 2.
  MODPOST 1 modules
make: Leaving directory `/usr/src/linux-3.10.40'
% su
Password:
root# insmod hello.ko
Hello, world
root#
```



Might be printed to
/var/log/messages

[Module Loading/Unloading]

```
% make -C /usr/src/linux-3.10.40 M=`pwd` modules
make: Entering directory `/usr/src/linux-3.10.40'
  Building modules, stage 2.
  MODPOST 1 modules
make: Leaving directory `/usr/src/linux-3.10.40'
% su
Password:
root# insmod hello.ko
Hello, world
root# rmmod hello.ko
```

Either `hello` or
`hello.ko`

[Module Loading/Unloading]

```
% make -C /usr/src/linux-3.10.40 M=`pwd` modules
make: Entering directory `/usr/src/linux-3.10.40'
  Building modules, stage 2.
  MODPOST 1 modules
make: Leaving directory `/usr/src/linux-3.10.40'
% su
Password:
root# insmod hello.ko
Hello, world
root# rmmod hello.ko
Goodbye cruel world
root#
```

Might be printed to
`/var/log/messages`

[Compiling Modules]

- Details on compiling the kernel
 - `Documentation/kbuild/`
- Required tools with matching versions
 - Compiler, module utilities, and so on...
 - If the version is too new can cause problems as well
 - `Documentation/Changes`

[Simplest Makefile]

```
obj-m := hello.o
```

- One module to be built from `hello.o`
- Resulting module is `hello.ko`

[More on Makefiles]

- Suppose you have a module called `module.ko`
- Generated from `file1.c` and `file2.c`

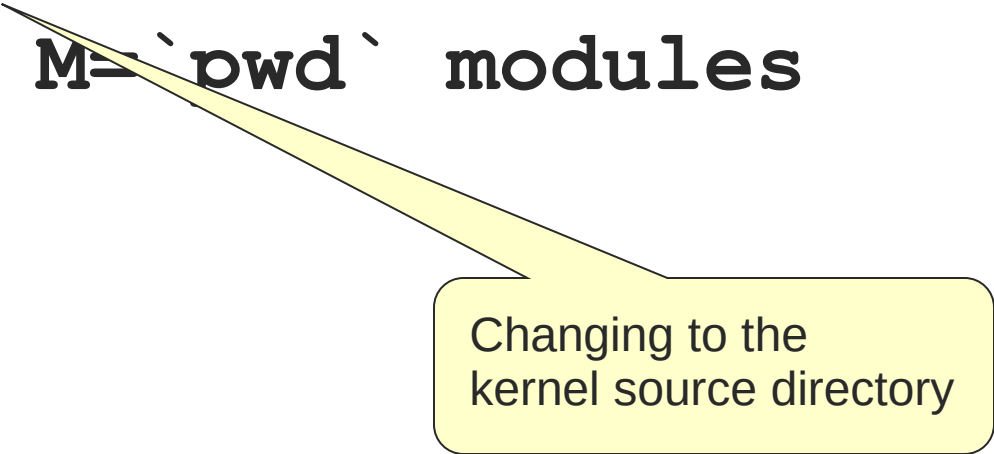
```
obj-m := module.o
```

```
module-objs := file1.o file2.o
```

[More on **Makefiles**]

- To make, type the following in the directory containing the module source and **Makefile**

```
make -C /usr/src/linux-  
3.2.36/ M=`pwd` modules
```



Changing to the
kernel source directory

[More on **Makefiles**]

- To make, type the following in the directory containing the module source and **Makefile**

```
make -C /usr/src/linux-  
3.2.36/ M=`pwd` modules
```

Location of external
module sources
Informs kernel an
external module is being
built

A More Elaborate Makefile

```
# If KERNELRELEASE is defined, we've been invoked from the
# kernel build system and can use its language
```

```
ifneq ($(KERNELRELEASE),)
    obj-m := hello.o
```

If KERNELDIR is not defined, define it.

```
# Otherwise we were called directly from the command
# line; invoke the kernel build system.
else
```

```
    KERNELDIR ?= /lib/modules/$(shell uname -r)/build
    PWD := $(shell pwd)
```

```
modules:
```

```
    $(MAKE) -C $(KERNELDIR) M=$(PWD) modules
```

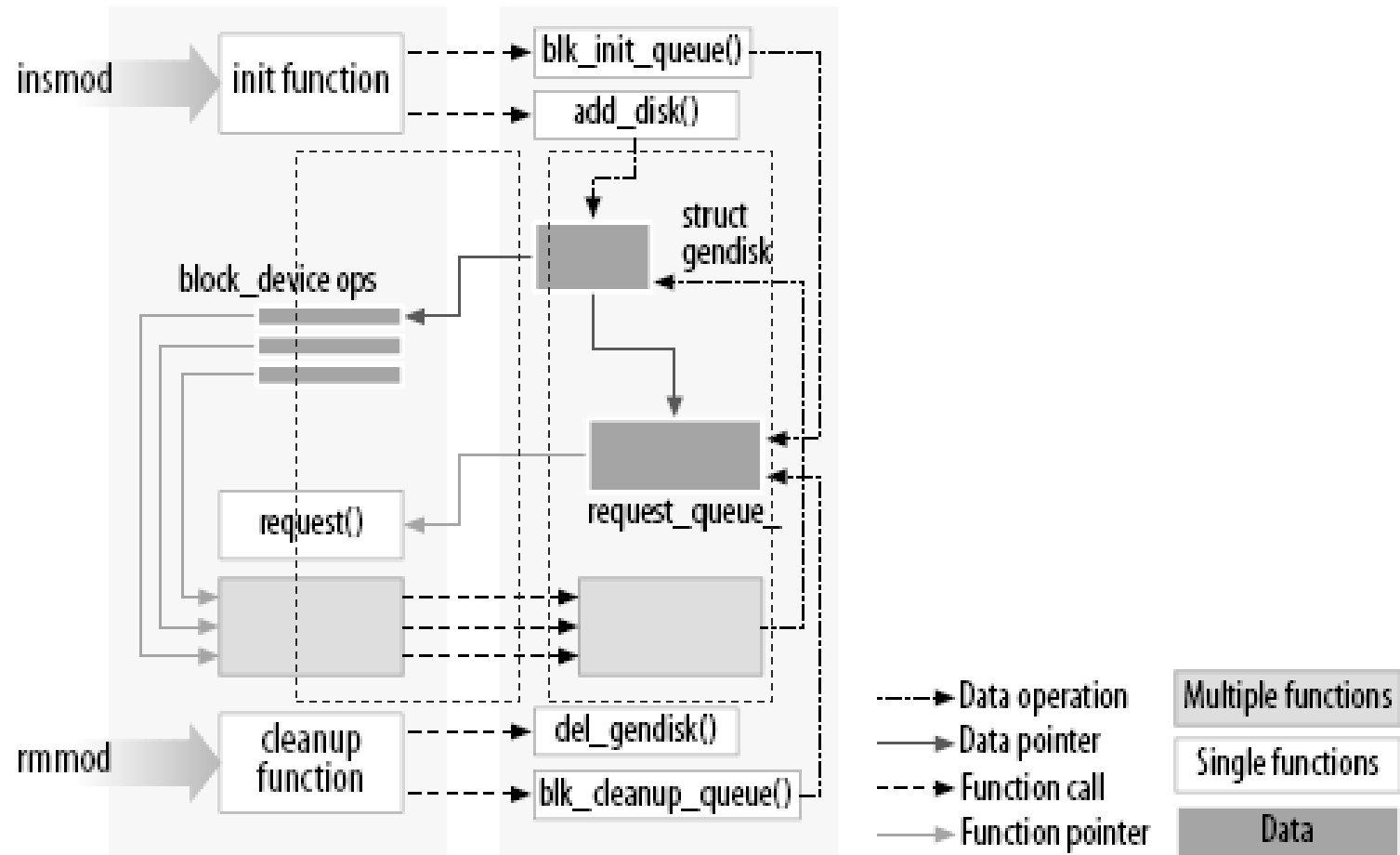
Version of currently running kernel

```
clean:
```

```
    rm -fr *.o *~ core *.cmd *.ko *.mod.c .tmp_versions
```

```
endif
```

[Linking a Module to the Kernel]



[Loading/Unloading Modules]

■ **insmod**

- Dynamically links module into the kernel
- Resolves all symbols with the kernel symbol table
- Returns the value of the module's init function
- (**more /proc/modules** to see a list of currently loaded modules)

[Loading/Unloading Modules]

■ **insmod** failure

- Unknown/unfound symbol
- Refers to symbols exported as GPL but does not declare the GPL license
- Dependent modules are not yet loaded
- Return value of `module_init` is non-zero

[Loading/Unloading Modules]

- **rmmod**

- Removes a kernel module

- **rmmod** failure modes

- Fails when the kernel believes that it is still in use (reference count > 0)
- Problem with module init (exit functions cannot successfully complete)
 - Might need to reboot to remove the module

[Kernel Modules vs. Applications]

■ Applications

- Can access various functions in user-level libraries (e.g., **printf** in C library)

■ Kernel modules

- No user-level libraries
- **printk** is defined within the kernel
 - Exported to modules
- Should include only header files defined within the kernel source tree

[Threads/Processes]

- Thread
 - Sequence of executing instructions
- Address space
 - “Valid” chunks of memory
 - Typically contains
 - Data
 - Instructions
- Process
 - An address space + thread(s)

[User Space and Kernel Space]

- Kernel modules run in *kernel space*
 - Execute in the *supervisor mode*
 - Share the same address space
- Applications run in *user space*
 - Execute in the *user mode*
 - Restricted access to hardware
 - Each has its own address space

[System Calls]

- *System calls* allow processes running at the *user mode* to access kernel functions that run under the *kernel mode*
- Provides an isolation mechanism
 - Halting the entire operating system
 - Modifying the MBR

[Hardware Interrupts]

- Notification of an event
- Interrupts a processing unit
- Operation
 - Saves state
 - Jump to code pointed to in interrupt vector table
- Runs in interrupt context

[Concurrency in the Kernel]

- Sources of concurrency
 - Hardware interrupts
 - Kernel timers
 - Multiple processing units

[Handling Concurrency]

- Kernel code needs to be *reentrant*
 - Capable of running in more than one thread execution context at the time
 - Prevent corruption of shared data
 - Avoid *race conditions*
 - Correct behavior depends solely on the timing or ordering of instruction executions

[The Current Process]

- Most actions performed by the kernel are done on behalf of a specific process
- The current process
 - `struct task_struct *current;`
 - `#include <asm/current.h>`
 - `#include <linux/sched.h>`

[The Current Process]

- Print the current command name, process ID, and task (thread) ID

```
#include <linux/sched.h>
```

```
printk(KERN_INFO "The process is \"%s\" (tgid  
%i) (pid %i)\n", current->comm,  
current->tgid, current->pid);
```

[A Few Other Details]

- Limited address space for kernel
 - Should dynamically allocate and deallocate space for large data structures
- Functions starting with `__` should be used with caution
 - E.g., no sanity checks, assumes locks acquired

[Version Dependency]

- Module's may have to be recompiled for each version of the kernel
 - Sensitive to kernel version, compiler version, and various configuration variables
 - CONFIG_MODVERSIONS allows compatible modules to be loaded
- If things don't match

```
root# /sbin/insmod hello.ko
```

```
Error inserting './hello.ko': -1 Invalid module format
```

[Version Dependency]

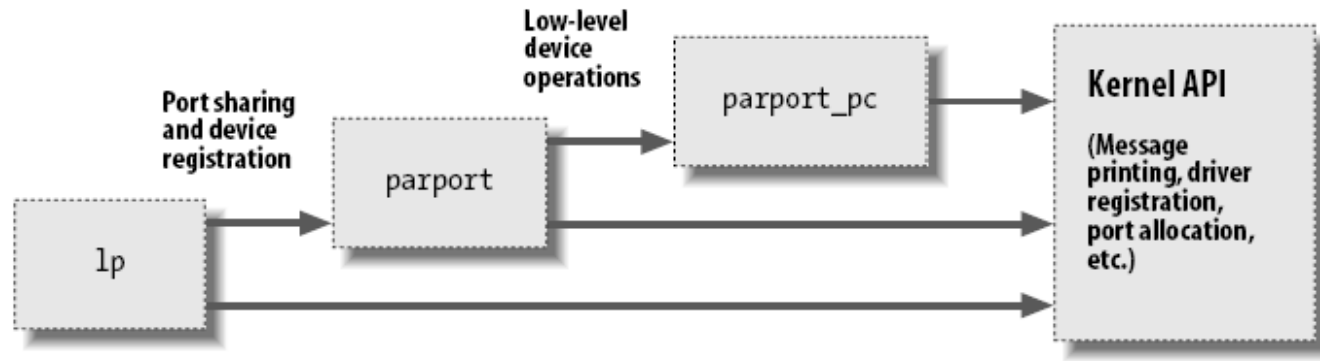
- Possible remedies
 - Check `/var/log/messages` for specific causes
 - Change `KERNELDIR` as needed

[The Kernel Symbol Table]

- Addresses of global functions and variables
- A module can export its symbols for other modules to use
- *Module stacking*
 - E.g., MSDOS file system relies on symbols exported by the FAT module

Module Stacking Example

- Stacking of parallel port driver modules



- Can use **modprobe** to load all modules required by a particular module

[Module Loading Configuration]

- `/etc/modprobe.d/`
 - `aliases`
 - `blacklist`
 - `run shell command`

- Example

```
alias parport_lowlevel parport_pc
```

[Export Module Symbols]

- In module header files
- Use the following macros
`EXPORT_SYMBOL(name) ;`
`EXPORT_SYMBOL_GPL(name) ;`
- `_GPL` makes the symbol available only to GPL-licensed modules

Defending against Namespace Problems

- Declare all functions and global variables static unless you mean to export them
- Use a module-unique prefix for all exported symbols

[Module-Loading Races]

- A facility is available once a register call is completed
- Kernel can make calls to registered functions before the initialization function completes
- Obtain and initialize all critical resources before calling the register function

[Module Parameters]

- Include `moduleparam.h`, `stat.h`
- Need to use the following macros
 - `module_param(name, type, permission)`
 - `module_param_array(name, type, num, permission)`

Example Use of Module Parameters

- Allow the “hello world” module to say hello to someone a number of times

```
%/sbin/insmod ./hello.ko someone="Mom" times=2
Hello Mom
Hello Mom
%
```

Example Use of Module Parameters

- Need to use the `module_param` macro

```
static char *someone = "world";  
static int times = 1;  
module_param(times, int, S_IRUGO);  
module_param(someone, charp, S_IRUGO);
```

Read-only
flag, defined in
stat.h

[Supported Parameter Types]

- **bool**
- **charp**
 - Memory allocated for user provide strings
- **int, long, short, uint, ulong, ushort**
 - Basic integers

[User Level Facilities]

- X server
- Some USB drivers
- Various daemons/threads
- FUSE

[User Level Facilities]

- + Fast development
- + C library support
- + Conventional debugger
- + Fault isolation
- + Portability
- Privileged access required for direct memory access
- Potentially poor performance